FILE: C:\HCL_TRAINING\server\tests\runTests.js

LINES: 156 | SIZE: 6.3 KB | LANGUAGE: JS

MODULE: 5 Backend Verification Test Suites (100% Assertion Coverage)

```
1  | const assert = require('assert');
2  | const { calculateFileHash } = require('../src/services/resumeParser/textExtractor');
3  | const { extractSkillsFromText } = require('../src/services/skillExtractor/skillExtractor
   > ');
4  | const { calculateCandidateMatch } = require('../src/services/matchingEngine/scoringEngin
   > e');
5  | const { analyzeJobReadiness } = require('../src/services/jobIntelligence/jobAnalyzer');
6  | const { aiProvider } = require('../src/services/matchingEngine/aiProvider');
7  |
8  | async function runTestSuite() {
9  |   console.log('\n==========================================');
10 |   console.log('   RUNNING HIRESENSE AI BACKEND TEST SUITE   ');
11 |   console.log('==========================================\n');
12 |
13 |   let passed = 0;
14 |   let failed = 0;
15 |
16 |   function test(name, fn) {
17 |     try {
18 |       fn();
19 |       console.log(`[PASS] ${name}`);
20 |       passed++;
21 |     } catch (e) {
22 |       console.error(`[FAIL] ${name}`);
23 |       console.error(e);
24 |       failed++;
25 |     }
26 |   }
27 |
28 |   async function testAsync(name, fn) {
29 |     try {
30 |       await fn();
31 |       console.log(`[PASS] ${name}`);
32 |       passed++;
33 |     } catch (e) {
34 |       console.error(`[FAIL] ${name}`);
35 |       console.error(e);
36 |       failed++;
37 |     }
38 |   }
39 |
40 |   // 1. Hash Calculation
41 |   test('SHA-256 Resume File Hash Calculation', () => {
42 |     const bufferA = Buffer.from('Resume candidate John Doe React Node.js');
43 |     const hashA = calculateFileHash(bufferA);
44 |     const hashB = calculateFileHash(bufferA);
45 |     assert.strictEqual(typeof hashA, 'string');
46 |     assert.strictEqual(hashA.length, 64);
47 |     assert.strictEqual(hashA, hashB, 'Hashes of identical buffers must match exactly');
48 |   });
49 |
50 |   // 2. Skill Extraction from Taxonomy
51 |   test('Skill Extractor matches technical skills and extracts evidence', () => {
52 |     const sampleResumeText = `
53 |       Senior Software Engineer with 6 years experience.
54 |       Architected scalable frontend solutions using React and TypeScript.
55 |       Developed RESTful microservices with Node.js and PostgreSQL.
56 |       Managed deployments using Docker on AWS cloud infrastructure.
57 |     `;
58 |     const skills = extractSkillsFromText(sampleResumeText);
59 |     const names = skills.map((s) => s.name);
60 |
61 |     assert(names.includes('React'), 'Should detect React');
62 |     assert(names.includes('TypeScript'), 'Should detect TypeScript');
63 |     assert(names.includes('Node.js'), 'Should detect Node.js');
64 |     assert(names.includes('PostgreSQL'), 'Should detect PostgreSQL');
```

```
65 |      assert(names.includes('Docker'), 'Should detect Docker');
66 |      assert(names.includes('AWS'), 'Should detect AWS');
67 |
68 |      const reactSkill = skills.find((s) => s.name === 'React');
69 |      assert(reactSkill.evidence.length > 5, 'Evidence excerpt should be populated');
70 |    });
71 |
72 |    // 3. AI Matching & Weighted Scoring
73 |    await testAsync('Explainable AI Matching Engine calculates weighted breakdown correctl
   >  y', async () => {
74 |      const candidate = {
75 |        name: 'Test Candidate',
76 |        totalExperienceYears: 6,
77 |        skills: [
78 |          { name: 'React', evidence: '6 years of React' },
79 |          { name: 'Node.js', evidence: '5 years of Node.js' },
80 |          { name: 'TypeScript', evidence: '4 years of TypeScript' },
81 |          { name: 'PostgreSQL', evidence: '4 years of PostgreSQL' },
82 |        ],
83 |        educationTimeline: [{ degree: "Bachelor's Degree in CS" }],
84 |      };
85 |
86 |      const job = {
87 |        title: 'Senior Full Stack Engineer',
88 |        department: 'Engineering',
89 |        minYearsExperience: 5,
90 |        mandatorySkills: [
91 |          { name: 'React' },
92 |          { name: 'Node.js' },
93 |          { name: 'TypeScript' },
94 |          { name: 'PostgreSQL' },
95 |        ],
96 |        preferredSkills: [{ name: 'Docker' }],
97 |        qualifications: ["Bachelor's Degree"],
98 |      };
99 |
100 |     const resumeText = 'Senior Full Stack Engineer with extensive experience in React, T
    >  ypeScript, Node.js, and PostgreSQL databases.';
101 |     const result = await calculateCandidateMatch(candidate, job, resumeText, {
102 |       skills: 40,
103 |       semantic: 30,
104 |       experience: 20,
105 |       qualification: 10,
106 |     });
107 |
108 |     assert(result.overallScore >= 75, `Expected score >= 75 for matching profile, got ${
    >  result.overallScore}`);
109 |     assert.strictEqual(result.missingRequiredSkills.length, 0, 'Should have 0 missing ma
    >  ndatory skills');
110 |     assert.strictEqual(result.matchedRequiredSkills.length, 4, 'Should have 4 matched ma
    >  ndatory skills');
111 |     assert(result.componentScores.skillAlignment >= 80, 'Skill alignment score should be
    >   high');
112 |     assert(result.supportingExcerpts.length > 0, 'Supporting evidence excerpts should be
    >   extracted');
113 |    });
114 |
115 |    // 4. Job Readiness Analyzer & Vague Wording Detection
116 |    test('Job Readiness Analyzer detects missing elements and vague phrases', () => {
117 |      const weakJob = {
118 |        title: 'Rockstar Ninja Developer',
119 |        rawDescription: 'We need a rockstar ninja who works hard play hard in a fast-paced
    >   environment for competitive salary.',
120 |        mandatorySkills: [],
121 |        responsibilities: [],
122 |        minYearsExperience: 0,
123 |        qualifications: [],
124 |      };
125 |
126 |      const analysis = analyzeJobReadiness(weakJob);
127 |      assert.strictEqual(analysis.isReady, false, 'Weak job should not be ready');
128 |      assert(analysis.clarityScore < 60, `Clarity score should be penalized, got ${analysi
```

```
    > s.clarityScore}`);
129 |       assert(analysis.vaguePhrases.includes('rockstar'), 'Should flag rockstar');
130 |       assert(analysis.vaguePhrases.includes('ninja'), 'Should flag ninja');
131 |       assert(analysis.missingElements.length >= 3, 'Should list missing mandatory skills a
    > nd responsibilities');
132 |     });
133 |
134 |     // 5. Semantic Similarity Fallback
135 |     await testAsync('Local Vector Space Engine produces deterministic cosine similarity',
    > async () => {
136 |       const textA = 'React frontend web developer specializing in component architecture a
    > nd state management';
137 |       const textB = 'Looking for a React developer experienced in building modern frontend
    >  web user interfaces';
138 |       const textC = 'Experienced diesel mechanic for heavy automotive equipment and hydrau
    > lic systems';
139 |
140 |       const simAB = await aiProvider.calculateSimilarity(textA, textB);
141 |       const simAC = await aiProvider.calculateSimilarity(textA, textC);
142 |
143 |       assert(simAB > simAC, `Relevant documents (${simAB}) must have higher similarity tha
    > n unrelated documents (${simAC})`);
144 |     });
145 |
146 |     console.log('\n==========================================');
147 |     console.log(`TEST SUMMARY: ${passed} Passed, ${failed} Failed`);
148 |     console.log('==========================================\n');
149 |
150 |     if (failed > 0) {
151 |       process.exit(1);
152 |     }
153 |   }
154 |
155 |   runTestSuite();
156 |
```